declarations in a single file is specifically prohibited by the PSR-1 coding standard (section three, to be precise).

On the other hand, explicitly requiring every file containing a class declaration also becomes problematic, not to mention wasteful of system resources. If defining classes on a one-per-file basis is going to work in a frictionless way, we need to automate it somehow. This is where autoloading comes in.

Autoloading

In general programming terms, autoloading simply means that sections of a program can be fetched from storage when needed, rather than being explicitly imported. In PHP, autoloading is specific to classes and the other class-like constructs (`interface`, `trait`, and `enum`). To enable this mechanism, you must create a function that is able to `require` or `include` the file containing the missing declaration. This loading function must then be registered with PHP's `spl_autoload_register()` function. Here is a quick example:

```
spl_autoload_register(function ($className) {
    $fileName = '10.01_' . $className . '.php';
    print "{$className} is not declared, attempting to load from ./
    {$fileName}" . PHP_EOL;
    require __DIR__ . DIRECTORY_SEPARATOR . $fileName;
});

$someClass = new SomeClass();
var_dump($someClass);

Output:
SomeClass is not declared, attempting to load from ./10.01_SomeClass.php
object(SomeClass)#2 (0) {
}
```

This is autoloading in its simplest form. In reality, the mapping will be slightly more complex because classes that are distributed in a namespace hierarchy are also distributed across a filesystem. The good news is that thanks to PSR-4, all we need to do is transform PHP's namespace path into a filesystem path (directory separator characters vary across systems, but the namespace separator is always a backslash) and know where the top-level namespace exists on the filesystem.